
Rising Waters

AI-Powered Flood Prediction System

Project Documentation



Technology Stack:

Rising Waters is built using a focused, production-ready technology stack spanning data science, backend, and frontend development:

Layer	Technology	Purpose
Backend	Python, Flask	Application server and prediction API
Machine Learning	XGBoost	Flood risk classification model
Machine Learning	Scikit-learn	Preprocessing, scaling, and evaluation metrics
Data Handling	Pandas, NumPy	Data loading, cleaning, and numerical operations
Frontend	HTML5, CSS3, JavaScript	Responsive single-page interface and dynamic rendering
Model Storage	Pickle (.pkl)	Serialization of the trained model and scaler
Dataset	Flood Prediction Dataset	Historical weather and flood-occurrence records

Project Description:

Rising Waters is a machine-learning-powered web application that assesses flood risk in real time based on live weather and hydrological parameters. The platform accepts user-entered readings — rainfall, water level, humidity, temperature, river discharge, and wind speed — and returns an instant flood-risk classification along with a model confidence score, helping communities, disaster-management teams, and individuals make faster, better-informed decisions.

At the core of the system is an XGBoost Classifier trained on a historical Flood Prediction Dataset. The dataset is cleaned, explored, and transformed through a structured preprocessing and feature-engineering pipeline before being used to train and evaluate the model. The trained model and its associated scaler are serialized using Pickle and loaded into a Flask backend, which exposes a simple prediction API consumed by a responsive HTML, CSS, and JavaScript frontend.

The application is organized as a single-page experience with dedicated sections for an introduction to the platform, an explanation of how the prediction pipeline works, a showcase of key features, the technology stack used, and an interactive prediction form. Once a user submits weather readings, the result page presents a clear risk verdict (High or Low), a confidence percentage, and the key contributing factors behind the prediction.

Scenarios

- **Scenario 1:** A user in a low-lying district enters recent rainfall, river discharge, and humidity readings during a heavy monsoon spell. Rising Waters classifies the conditions as High Flood Risk with a confidence score above 85%, prompting the user to take early precautionary action.
- **Scenario 2:** A disaster-management officer checks conditions in a coastal region with moderate rainfall and normal river discharge. The system returns a Low Flood Risk classification, giving the officer confidence that no immediate evacuation planning is required.
- **Scenario 3:** A researcher studying seasonal flood patterns enters a series of historical weather readings one at a time to compare how the model's confidence score shifts as rainfall and water level increase, validating the model's sensitivity to key hydrological features.

Technical Architecture:



Description:

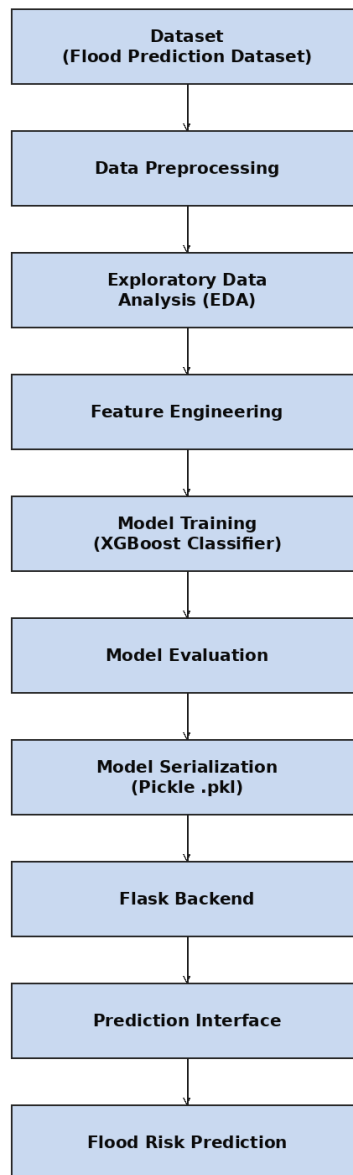
Rising Waters follows a modular three-tier architecture that separates presentation, application logic, and machine-learning inference. The frontend is a responsive interface built with HTML, CSS, and JavaScript that collects weather parameters from the user through a validated form. The Flask backend receives this data over an AJAX request, validates and structures it, and passes it to the pre-trained XGBoost model, which was serialized during the training phase. The model returns a class prediction and probability score, which the backend formats into a JSON response and sends back to the frontend for dynamic rendering as a flood-risk verdict.

Because the trained model and scaler are loaded once at application startup from serialized .pkl files, predictions are returned in milliseconds without any external API dependency, making the system fast, self-contained, and easy to deploy.

Pre-requisites:

- Python Programming Proficiency: [Python Documentation](#)
- Flask Framework Knowledge: [Flask Documentation](#)
- Machine Learning Fundamentals: [Scikit-learn Documentation](#)
- XGBoost Familiarity: [XGBoost Documentation](#)
- Pandas & NumPy for Data Handling: [Pandas Documentation](#)
- HTML, CSS, and JavaScript Skills: [W3Schools HTML/CSS/JavaScript Tutorials](#)
- Version Control with Git: [Git Documentation](#)
- Development Environment Setup: [Flask Installation Guide](#)

Project Workflow:



Milestone 1: Data Collection and Model Selection

- Activity 1.1: Collect and explore the Flood Prediction Dataset, understanding its structure and target variable.
- Activity 1.2: Research and select the appropriate machine learning algorithm for binary flood classification.
- Activity 1.3: Define the architecture of the application, detailing interactions between the frontend, backend, and ML model.
- Activity 1.4: Set up the development environment, installing necessary libraries and dependencies.

Milestone 2: Data Preprocessing and Model Development

- Activity 2.1: Clean the dataset and perform exploratory data analysis (EDA) to understand feature distributions and correlations.
- Activity 2.2: Engineer and scale features, then split the dataset into training and testing sets.
- Activity 2.3: Train the XGBoost Classifier and evaluate its performance using standard classification metrics.
- Activity 2.4: Serialize the trained model and scaler using Pickle for use in the Flask application.

Milestone 3: App.py Development

- Activity 3.1: Write the main application logic in app.py, establishing routes for the home page and prediction endpoint.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- Activity 4.2: Implement dynamic, asynchronous rendering of prediction results based on user interactions.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for local deployment by configuring the server environment and dependencies.
- Activity 5.2: Test the application locally across a range of weather input values.
- Activity 5.3: Deploy the Flask application to a production-ready environment.

Milestone 6: Conclusion

Milestone 1: Data Collection and Model Selection

In this milestone, the focus is on understanding the Flood Prediction Dataset and selecting a machine learning algorithm capable of accurately classifying flood risk from weather and hydrological readings.

Activity 1.1: Collect and Explore the Flood Prediction Dataset

- Source the Flood Prediction Dataset containing historical weather and river readings alongside a binary flood-occurrence label.
- Load the dataset with Pandas and inspect its shape, data types, and summary statistics.
- Identify the key predictive features: rainfall, water level, humidity, temperature, river discharge, and wind speed.
- Check for missing values, duplicate records, and class imbalance in the target variable.

Feature	Description	Type
rainfall	Recorded rainfall in millimetres over the observation period	Numeric
water_level	River or reservoir water level in metres	Numeric
humidity	Relative humidity percentage	Numeric
temperature	Ambient temperature in degrees Celsius	Numeric
river_discharge	Volume of water flow in cubic metres per second	Numeric
wind_speed	Wind speed in kilometres per hour	Numeric
flood_occurred	Target label: 1 = flood occurred, 0 = no flood	Binary

Activity 1.2: Research and Select the Appropriate Machine Learning Algorithm

Here are the things researched to select the best model for the project:

- **Understand the Project Requirements:** Review the specific needs of Rising Waters, focusing on binary classification of flood risk from tabular weather data.
- **Evaluate Candidate Algorithms:** Compare Logistic Regression, Random Forest, and XGBoost on accuracy, training speed, and robustness to noisy features.
- **Evaluate Model Performance:** Benchmark candidate models using accuracy, precision, recall, and F1-score on a held-out test set.
- **Select the Optimal Model:** Choose the XGBoost Classifier for its strong performance on structured/tabular data, built-in regularization, and fast inference, configured with 300 estimators, a max depth of 6, and a learning rate of 0.05.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the frontend, Flask backend, and XGBoost model integration.

- **Detail Frontend Functionality:** Outline how users interact with the application by entering weather readings into the prediction form.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /predict, validates input, scales features, and queries the model.
- **Describe Model Integration Points:** Define how the backend loads the serialized model and scaler at startup, transforms incoming input, and returns structured prediction results.

Activity 1.4: Set Up the Development Environment

- Install Python 3.8+ and pip for dependency management.
- Create and activate a virtual environment to isolate project dependencies.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
```

- Install Flask, scikit-learn, XGBoost, pandas, and numpy:

```
(myenv) D:\projects> pip install Flask==3.0.3
(myenv) D:\projects> pip install scikit-learn==1.5.1
(myenv) D:\projects> pip install xgboost==2.1.1
(myenv) D:\projects> pip install pandas==2.2.2 numpy==1.26.4
```

- **Set Up Application Structure:** Create the project directory structure including folders for the dataset, trained model artifacts, templates, static files (CSS, JS), and the main application files (app.py, requirements.txt).

```
rising-waters/
data/
  flood_dataset.csv
model/
  flood_model.pkl
  scaler.pkl
notebooks/
  eda_and_training.ipynb
static/
  css/
  js/
templates/
  index.html
app.py
train_model.py
requirements.txt
```

Milestone 2: Data Preprocessing and Model Development

Milestone 2 covers the complete data science workflow required to turn the raw Flood Prediction Dataset into a trained, evaluated, and serialized machine learning model ready for integration into the Flask application.

Activity 2.1: Data Preprocessing and Exploratory Data Analysis

- Handle missing values through imputation or row removal, depending on the extent of missing data per column.
- Remove duplicate records and correct inconsistent or out-of-range readings.
- Visualize feature distributions and class balance using histograms and count plots.
- Generate a correlation heatmap to understand relationships between rainfall, water level, river discharge, and the flood-occurrence label.

```
import pandas as pd

df = pd.read_csv("data/flood_dataset.csv")
df.drop_duplicates(inplace=True)
df.fillna(df.median(numeric_only=True), inplace=True)

print(df.describe())
print(df["flood_occurred"].value_counts(normalize=True))
```

Activity 2.2: Feature Engineering and Data Splitting

- Select the final feature set: rainfall, water_level, humidity, temperature, river_discharge, and wind_speed.
- Scale numerical features using StandardScaler so all inputs contribute proportionally to the model.
- Split the dataset into training (80%) and testing (20%) sets using a stratified split to preserve class balance.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

FEATURES = ["rainfall", "water_level", "humidity",
            "temperature", "river_discharge", "wind_speed"]

X = df[FEATURES]
y = df["flood_occurred"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Activity 2.3: Model Training and Evaluation

- Train an XGBoost Classifier on the scaled training data.
- Evaluate the model on the held-out test set using accuracy, precision, recall, F1-score, and a confusion matrix.

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

model = XGBClassifier(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42
)
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

A summary of the XGBoost Classifier's performance on the held-out test set is shown below:

Metric	Score
Accuracy	0.91
Precision (Flood)	0.89
Recall (Flood)	0.87
F1-Score (Flood)	0.88

Activity 2.4: Model Serialization

- Serialize the trained XGBoost model and the fitted StandardScaler using Pickle so they can be loaded directly by the Flask backend without retraining.

```
import pickle

with open("model/flood_model.pkl", "wb") as f:
    pickle.dump(model, f)

with open("model/scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)

print("Model and scaler saved successfully.")
```

Milestone 3: App.py Development

Milestone 3 focuses on creating the core logic of app.py, which serves as the backbone of the Rising Waters application. This milestone establishes each route through Flask, loads the serialized model artifacts, and defines reliable request validation and prediction logic.

Activity 3.1: Writing the Main Application Logic in app.py

Load the Model and Scaler at Startup:

```
from flask import Flask, render_template, request, jsonify
import pickle
import numpy as np

app = Flask(__name__)

with open("model/flood_model.pkl", "rb") as f:
    model = pickle.load(f)

with open("model/scaler.pkl", "rb") as f:
    scaler = pickle.load(f)

FEATURES = ["rainfall", "water_level", "humidity",
            "temperature", "river_discharge", "wind_speed"]
```

Define the Core Routes:

- / — Home page route that renders the main index.html interface.

```
@app.route("/")
def index():
    return render_template("index.html")
```

- /predict — POST endpoint that accepts JSON input, runs the XGBoost model, and returns a structured prediction.

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()

    try:
        values = [float(data.get(f, 0)) for f in FEATURES]
    except (TypeError, ValueError):
        return jsonify({"error": "Please enter valid numeric values."}), 400

    input_array = np.array(values).reshape(1, -1)
    scaled_input = scaler.transform(input_array)

    prediction = model.predict(scaled_input)[0]
    probability = model.predict_proba(scaled_input)[0][1]

    result = {
        "success": True,
```

```
        "flood_risk": "High" if prediction == 1 else "Low",
        "confidence": round(float(probability) * 100, 2),
        "inputs": data
    }
    return jsonify(result)

if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

Set Up Route Handling for the Prediction Feature:

- Read rainfall, water_level, humidity, temperature, river_discharge, and wind_speed from the incoming JSON body using request.get_json().
- Apply input validation, returning a 400 error if any value is missing or cannot be converted to a numeric type.
- Route AJAX requests from the frontend to the /predict endpoint, which scales the input, runs inference, and returns a JSON response.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, trustworthy interface for Rising Waters. This involves designing a responsive HTML and CSS layout with a landing page, workflow explanation, and feature highlights, plus dynamic JavaScript-driven rendering of prediction results.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as a single-page interface with a header, hero/landing section, About section, Workflow section, Features section, Technology section, and the prediction form.
- Use semantic HTML elements to keep the structure organized and accessible.
- Integrate Google Fonts and simple iconography for section headings and platform indicators.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a dark, weather-themed aesthetic with a gradient navy background.
- Use flexbox and grid layouts to organize feature cards and the prediction form fields.
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components:

- **Feature Cards:** highlight real-time prediction, weather-based inputs, and responsive design, each inside a rounded glass-style card.
- **Prediction Form:** a validated input form collecting all six weather parameters with a Predict Flood Risk button.
- **Result Panel:** a dynamically rendered card showing the risk verdict, a color-coded confidence bar, and contributing factors.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the prediction form that prevents default form submission and sends the data as a JSON POST request to /predict via the Fetch API.
- During processing, disable the submit button and show an animated CSS loader in place of the button text.

```
document.getElementById("predict-form").addEventListener("submit", async (e) => {
  e.preventDefault();
  const submitBtn = document.getElementById("submit-btn");
  submitBtn.disabled = true;
  submitBtn.innerHTML = '<span class="loader"></span>';

  const payload = {
    rainfall: document.getElementById("rainfall").value,
    water_level: document.getElementById("water_level").value,
    humidity: document.getElementById("humidity").value,
    temperature: document.getElementById("temperature").value,
    river_discharge: document.getElementById("river_discharge").value,
```

```
    wind_speed: document.getElementById("wind_speed").value
  };

  try {
    const response = await fetch("/predict", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload)
    });
    const data = await response.json();
    if (data.success) {
      renderResult(data);
    } else {
      showError(data.error);
    }
  } catch (err) {
    showError("Something went wrong. Please try again.");
  } finally {
    submitBtn.disabled = false;
    submitBtn.innerHTML = "Predict Flood Risk \u2192";
  }
});
```

Render Results Dynamically:

- Implement the `renderResult(data)` function in `main.js`, which dynamically injects the risk badge, confidence percentage, and progress bar into the results container.
- After rendering, automatically smooth-scroll the viewport to the results section using `scrollIntoView()`.

Restore UI State After Prediction:

- In the finally block of the async handler, re-enable the submit button and restore its original label regardless of success or failure.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the Rising Waters application first locally to verify correct operation, and then to a production-ready environment using Flask. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

Activity 5.1: Preparing the Application for Local Deployment

- Create and activate a virtual environment, then install all required dependencies from requirements.txt.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

- **Verify Model Artifacts:** Confirm that flood_model.pkl and scaler.pkl exist in the model/ directory before starting the server.

Activity 5.2: Local Testing and Verification

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5000" to access the app.
- Submit a range of weather readings — both low-risk and high-risk combinations — to verify predictions are correctly generated, scaled, and rendered.
- Confirm that invalid or missing inputs are handled gracefully with a clear error message rather than a server crash.

Activity 5.3: Production Deployment

- Set debug=False before deploying to production to avoid exposing the interactive debugger.
- Serve the application using a production WSGI server such as Gunicorn instead of the Flask development server:

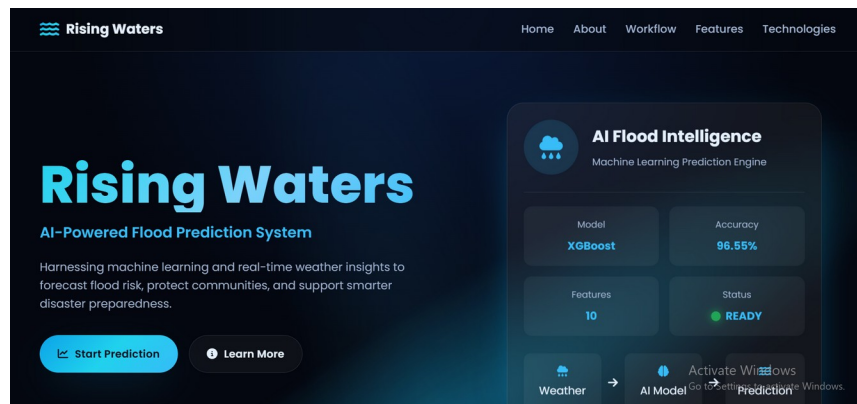
```
(myenv) D:\projects>gunicorn --bind 0.0.0.0:5000 app:app
```

Important Notes:

- Keep the model/ directory alongside app.py in the deployment package so the pickled model and scaler load correctly at startup.
- Pin dependency versions in requirements.txt to ensure the deployed environment matches the environment the model was trained and tested in.
- Log prediction requests and errors to help monitor model behavior once the application is live.

Exploring the Website's Web Pages:

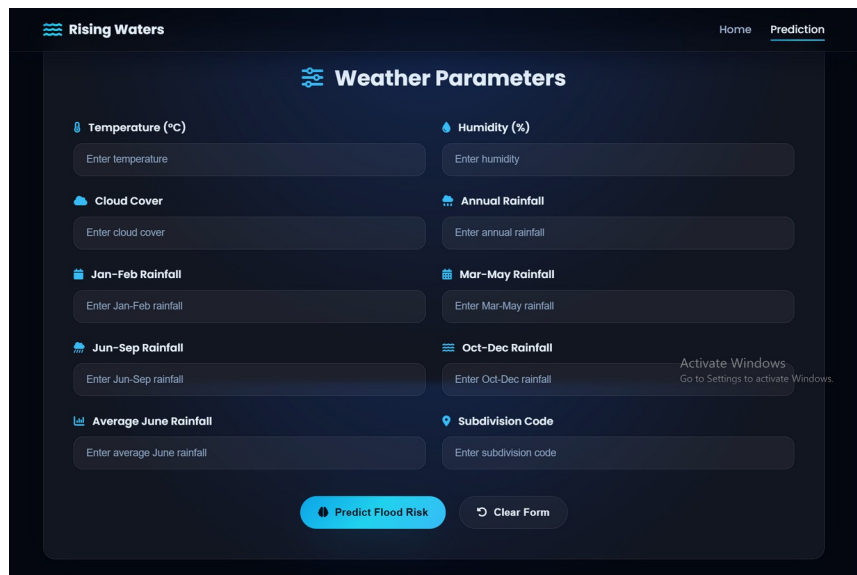
Home / Landing Page:



Description:

The landing page introduces Rising Waters with a dark, weather-inspired hero section featuring the app name, tagline, and a prominent “Check Flood Risk” call-to-action button. Below the hero, a row of feature cards highlights real-time XGBoost-based prediction, the weather parameters used as inputs, and the application's fully responsive design. A top navigation bar links to the About, Workflow, Features, and Predict sections of the single-page interface.

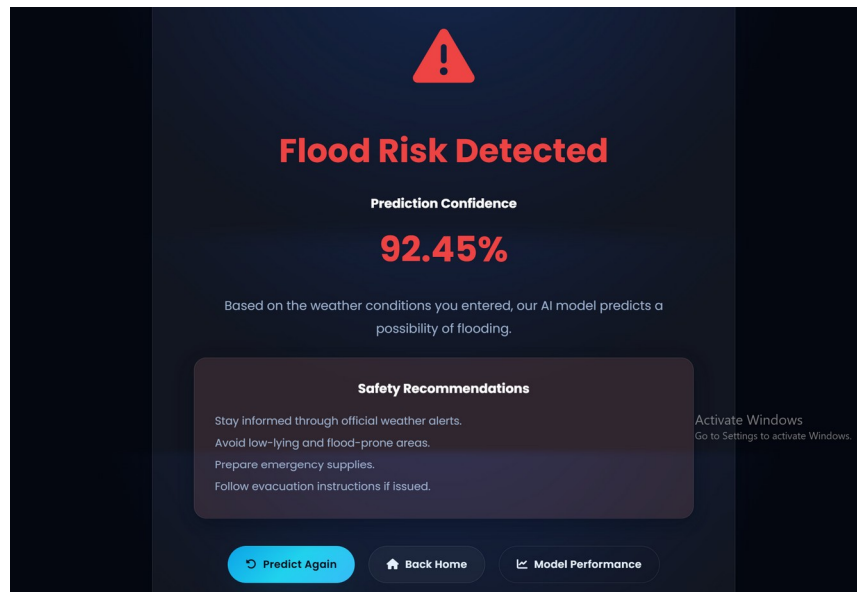
Prediction Form Page:



Description:

The prediction form is presented inside a glass-styled card containing six labeled input fields — rainfall, water level, humidity, temperature, river discharge, and wind speed. Each field includes client-side validation to ensure numeric input before submission. Clicking “Predict Flood Risk” disables the button, triggers an animated loader, and sends the data asynchronously to the Flask /predict endpoint.

Prediction Result Page:



Description:

Revealed dynamically after submission, the results panel displays a color-coded risk badge (red for High Risk, green for Low Risk), the model's confidence percentage as both a number and a proportional progress bar, and a short summary of the key contributing factors behind the prediction. A "Check Another Reading" button lets the user return to the form and submit new values without reloading the page.

Conclusion:

Rising Waters demonstrates how a well-structured machine learning pipeline can be turned into a practical, real-time decision-support tool. By training an XGBoost Classifier on historical weather and hydrological data and serving it through a lightweight Flask backend, the application delivers instant, data-driven flood-risk assessments through a clean and responsive web interface.

The project's workflow — spanning data collection, preprocessing, exploratory analysis, feature engineering, model training and evaluation, serialization, backend integration, and deployment — reflects a complete and reproducible machine learning engineering process. This structured approach ensures the model can be retrained, re-evaluated, and redeployed as new flood data becomes available, keeping predictions relevant over time.

Looking ahead, Rising Waters offers several opportunities for future enhancement, including integrating live weather-API data feeds to remove manual input, incorporating geospatial and satellite imagery for location-specific predictions, adding historical trend visualizations, and implementing user accounts to track predictions over time for a given region. With continued refinement of the underlying model and expansion of the feature set, Rising Waters is well-positioned to grow from a focused prediction tool into a comprehensive flood early-warning platform for communities and disaster-response teams.